4 Methods *(in a simple stand-alone Java program)*

Note! By default, there is a *line break* at the end of the output - if not explicitly stated otherwise.

Methods Numbers

Write a program called **MethodsNumbers** that includes a method called **printNumbers**. The **printNumbers** method should take the first and last integer to be printed as parameters. The method should then print all integers between the first integer and last integer. The *main method* should first input the first and last integer from the user and then call the **printNumbers** method with the inputted values.

Example output:

```
Enter first value: 1
Enter last value: 10

1 2 3 4 5 6 7 8 9 10
```

Methods Discount

Write a program called **MethodsDiscount** that includes a method called **computePercentage**. The **computePercentage** method should take list price and selling price as parameters. The method should compute and return the discount percentage.

The *main method* should first input the prices from the user and then call the **computePercentage** method with the inputted values. Finally, the *main method* should print the discount percentage with two decimal places. The **computePercentage** method should not print anything.

Example output:

```
Enter list price: 120,00
Enter selling price: 75,00

The discount percentage is 37,50 %
```

Methods Minimum

Write a program called **MethodsMinimum** that includes a method called **minimum** that takes three integers as parameters. The method should return the smallest of the three integers.

The *main method* should first input three integers from the user and then call the **minimum** method. Finally, the *main method* should print the value that the **minimum** method has returned. The **minimum** method should not print anything.

Example output:

```
Enter first integer: 2
Enter second integer: 2
Enter third integer: 1

The minimum value is 1
```

Methods Tools

Write a class called **MethodsTools** for storing generic methods. That is, the **MethodsTools** class should serve as *method library class* and should not contain the *main* method.

Copy your the existing code of the **minimum** method from the **MethodsMinimum** class to the **MethodsTools** class, and make the necessary modifications to ensure the method accessible from outside of the **MethodsTools** class.

Next, write a program called **MethodsToolsProgram** that uses the method **minimum** of the **MethodsTools** class. The *main method* should input three integers from the user, call the **minimum** method of the **MethodsTools** class to determine the minimum of the inputted values, and then print the minimum value.

Example output:

```
Enter first integer: 100
Enter second integer: 75
Enter third integer: 42

The minimum value is 42
```

NOTE! Viope instruction: Exercises with multiple source files

1. Copy/paste the source code of your program class to the Viope code editor window.
   - If there is a ***package*** directive, then remove it in Viope.
2. Copy/paste the other class definition to the Viope code editor window to the program class code.
   - Remove the ***public*** keyword from the class definition. Viope allows only the program class to be public.
3. If you have any ***import*** directives in your source texts, you should move them to the top of the Viope editor window.
   - Place them before the program class code.

Methods Triplex

Write a program called **MethodsTriplex** that includes a method called **triplex** that takes a positive integer as parameter and returns the *triplex* of the integer. The **triplex** method should not print anything.

In pseudo-mathematics,

- The *triplex* of 1 is 1.
- The *triplex* of a positive integer X greater than 1 is the product of X and every third positive integer less than X.
- E.g., triplex(5) is 1 x 4 x 5 = 20, triplex (7) is 1 x 4 x 7 = 28 , and triplex(8) is 1 x 4 x 7 x 8 = 224

The *main method* should first ask the user to enter a positive integer. Then, the *main method* should print all triplexes up to the inputted integer. The **main** method should call the **triplex** method to determine the triplex of each integer up to the inputted value. Please use a for loop in the main method.

Please use the for loop in the main method.

Example output:

```
Enter a positive integer: 8
triplex(1) = 1
triplex(2) = 2
triplex(3) = 3
triplex(4) = 4
triplex(5) = 20
triplex(6) = 24
triplex(7) = 28
triplex(8) = 224
```

Methods Primality (*This is a grade level 4-5 exercise*)

Write a program called **MethodsPrimality** that includes a method called **isPrime** that takes a positive integer as parameter and returns *true* if the integer is a prime. Otherwise, the method should return *false*. The **isPrime** method should not input anything from the user or print anything.

A *prime* is an integer greater than 1 that has no positive divisors other than 1 and itself.

The *main method* should first input a positive integer from the user. Then, the *main method* should print all non-primes that are smaller than the inputted integer. The *main method* should call the **isPrime** method to determine if an inputted integer is a prime or not.

If there are no non-primes that are smaller than the inputted integer, the main method should print "**Nothing to be printed**".

Please use the for loop in the main method.

Example output:

```
Enter a positive integer: 36
1 4 6 8 9 10 12 14 15 16 18 20 21 22 24 25 26 27 28 30 32 33 34 35
```

Note: There is no line break at the end of the output.

Methods Almanac (*This is a grade level 5 exercise*)

Write a program called **MethodsAlmanac** that includes a method called **printAlmanac**. The method should take the *month number* and the *year* as parameters and print a *one-month calendar in Finnish*. See the example output below for more details.

The *main method* should first input the *year* and *month number* from the user and then call the **printAlmanac** method.

Example output:

```
Enter year: 2025
Enter month: 2

 > Helmikuu 2025 <
 Ma Ti Ke To Pe La Su
                 1  2
  3  4  5  6  7  8  9
 10 11 12 13 14 15 16
 17 18 19 20 21 22 23
 24 25 26 27 28
```

INSTRUCTION (*please read the below carefully*)

You can use the *lookup array* below to display the month name based on the month number.

```
String[] months = {"Tammikuu", "Helmikuu", "Maaliskuu", "Huhtikuu", "Toukokuu",
    "Kesäkuu", "Heinäkuu", "Elokuu", "Syyskuu", "Lokakuu", "Marraskuu", "Joulukuu"};
```

This time, you should use the **LocalDate** class to create a date object. In the example below, the date is *1 December 2021*.

```
LocalDate myDate = LocalDate.of(2021, 12, 1);
```

You can determine the ***number of days in a month*** as shown below.

```
int daysInMonth = myDate.lengthOfMonth();
```

You can determine the ***day of week*** as shown below (1 = Monday , 2 = Tuesday, ...)

```
int dayOfWeek = myDate.getDayOfWeek().getValue();
```

In this exercise, the easiest way ***to print a day number*** is as shown below. The "**%3d**" *format specifier* means a minimum width of *3 spaces*, which will be right-justified.

```
System.out.printf("%3d", day);
```

NOTE: You can write one for loop to *print the first line of day numbers only*. Then you can write another loop to print the other lines of day numbers.

NOTE: Please notice that there is *one leading space* on each calendar line. That is, ***all*** *day numbers can be printed with the same format specifier*.